

Automating Code Review Activities by Large-Scale Pre-training

Zhiyu Li^{*†}
Peking University
China
AkinoLi@pku.edu.cn

Shuai Lu^{*}
Microsoft Research Asia
China
shuailu@microsoft.com

Daya Guo[†]
Sun Yat-sen University
China
guody5@mail2.sysu.edu.cn

Nan Duan[‡]
Microsoft Research Asia
China
nanduan@microsoft.com

Shailesh Jannu
LinkedIn
USA
sjannu@linkedin.com

Grant Jenks
LinkedIn
USA
gjenks@linkedin.com

Deep Majumder
LinkedIn
USA
dmajumder@linkedin.com

Jared Green
LinkedIn
USA
jagreen@linkedin.com

Alexey Svyatkovskiy
Microsoft DevDiv
USA
alsvyatk@microsoft.com

Shengyu Fu
Microsoft DevDiv
USA
shengyu@microsoft.com

Neel Sundaresan
Microsoft DevDiv
USA
neels@microsoft.com

ABSTRACT

Code review is an essential part to software development lifecycle since it aims at guaranteeing the quality of codes. Modern code review activities necessitate developers viewing, understanding and even running the programs to assess logic, functionality, latency, style and other factors. It turns out that developers have to spend far too much time reviewing the code of their peers. Accordingly, it is in significant demand to automate the code review process. In this research, we focus on utilizing pre-training techniques for the tasks in the code review scenario. We collect a large-scale dataset of real-world code changes and code reviews from open-source projects in nine of the most popular programming languages. To better understand code diffs and reviews, we propose CodeReviewer, a pre-trained model that utilizes four pre-training tasks tailored specifically for the code review scenario. To evaluate our model, we focus on three key tasks related to code review activities, including code change quality estimation, review comment generation and code refinement. Furthermore, we establish a high-quality benchmark dataset based on our collected data for these three tasks and conduct comprehensive experiments on it. The experimental results

demonstrate that our model outperforms the previous state-of-the-art pre-training approaches in all tasks. Further analysis show that our proposed pre-training tasks and the multilingual pre-training dataset benefit the model on the understanding of code changes and reviews.

CCS CONCEPTS

• **Software and its engineering** → **Automatic programming.**

KEYWORDS

Code review, deep learning, datasets, pre-training

ACM Reference Format:

Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, and Neel Sundaresan. 2022. Automating Code Review Activities by Large-Scale Pre-training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*, November 14–18, 2022, Singapore, Singapore. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3540250.3549081>

1 INTRODUCTION

Code review, a process of manually inspecting source code by teammates other than the code author, is recognized as a critical part of the software development lifecycle [13]. Studies have shown the huge benefits of code reviews [1, 3, 32, 33]. Compared with the traditional code review process formalized by Fagan in 1976 [12], which avoids introducing errors and defects but is cumbersome, modern code review activities involve fewer formal requirements and aim at fully understanding code changes [4]. Given its benefits, code review has been widely adopted in both open-source and industrial projects. As shown by Yang et al. [45], in open-source projects, such as Qt, there are ten thousand reviews taking place every month (~22k reviews in case of Qt). However, it's not free

^{*}Equal contribution.

[†]Work done during internship at Microsoft Research Asia.

[‡]Corresponding author is Nan Duan.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ESEC/FSE '22, November 14–18, 2022, Singapore, Singapore

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-9413-0/22/11...\$15.00

<https://doi.org/10.1145/3540250.3549081>

to take all advantages of code reviews. To make a correct judgement whether to accept the pull request, developers have to put much time and effort into code reviewing, checking code from every aspect, including logic, functionality, complexity, code style, documentation, etc. For example, Yang et al. [45] report that only 1,437 reviewers have given more than 1M reviews in 4 years in project Qt. Accordingly, it is in significant demand to automate the code review process.

Many researchers have explored ways to assist reviewers and committers (i.e., code authors) to reduce their workload in the code review process, such as recommending the best reviewer [9, 37], recommending or generating the possible review comments [18, 40, 41] and even revising the code before submitting it for review [40]. This paper shares the same goal to automate some specific tasks related to code review. We target three scenarios from both reviewers' and committers' perspectives. The first is called *code change quality estimation*, aiming to predict whether a code diff needs a review comment. It assists reviewers to pick a code diff chunk that might have issues among a large number of code chunks. The second task is *review generation* which can dramatically reduce the time cost for the reviewer. The last scenario is for committers, which is *code refinement* according to the previous code and reviewer's comment.

Motivated by the wide adaptation of deep learning (DL) techniques for software engineering and the rapid development of pre-training techniques, we leverage pre-training for automating code review activities. Recently, researchers have proposed many pre-trained models on source code [7, 14, 17, 43]. However, they can hardly handle the code review process. Chen et al. [7] propose Codex, a large scale pre-trained model based on GPT-3 [6]. Codex has been proven to be the most powerful model for code generation tasks. Since it is trained on the source code files in raw format, it knows very little about code review. We demonstrate that it cannot generate any meaningful comment in the *review generation* task based on our evaluation. Tufano et al. [40] attempt to use a pre-trained model for code review automation. However, their pre-training dataset is collected from Stack Overflow and CodeSearchNet [22], which is not directly related to code review process. Furthermore, they take the normal form of source code as the model input, ignoring the special format of code diff that can help the model better understand code changes.

To tackle the problems, we pre-train **CodeReviewer**, an encoder-decoder transformer model. Different from Tufano et al. [40]'s work, CodeReviewer is pre-trained on a large dataset in code review scenario, consisting of code diff hunks and code review comments. We propose four pre-training tasks, including diff tag prediction, denoising code diff, denoising review comment, and review comment generation to make CodeReviewer better understand code diffs and generate review comments. Figure 1 shows the overview of the workflow of our CodeReviewer.

To make CodeReviewer more suitable for the code review process, we build a large-scale dataset of code changes and corresponding review comments. Such code review data is collected from GitHub pull requests in high-quality projects with high stars covering nine of the most popular programming languages. Using GitHub code review data, we create pre-training and benchmark datasets for three tasks related to the code review process. To the

best of our knowledge, it is the largest multilingual code review dataset with complete information of code changes and reviews.

We further evaluate our CodeReviewer model on our benchmark dataset. Compare with previous state-of-the-art (SOTA) generative models for code, experimental results show that our model outperforms previous works on all three tasks. Further analysis proves the effectiveness of our proposed pre-training tasks and the multilingual high-quality dataset.

To summarize, the contributions of this work are:

- The first pre-trained model that takes code diffs as input in the code review scenario.
- Novel pre-training tasks for better code changes understanding and generation.
- A large-scale code review related pre-training dataset and a high-quality benchmark dataset for evaluation in nine programming languages.
- A comprehensive evaluation of CodeReviewer and previous SOTA models. Our dataset, code, and model are released ¹.

2 CODE REVIEW AUTOMATION TASKS

In this section, we give a formulated description of the code review process, provide the formal definitions of three key tasks abstracted from the code review process, and clarify the data format of the tasks. The employed symbols are listed in Table 1.

2.1 Code Review

In the code review process, contributors (code change author, P_C) update the source code to accomplish new features or fix bugs in the old version. The original code and updated code are denoted as C_0 and C_1 . Once the code changes ($D : C_0 \rightarrow C_1$) are ready for review, the author of these changes creates a pull request to start the code review process. Other peer contributors (reviewers, P_R) will review the code changes and provide comments or suggestions (R_{nl}) on them if necessary. Based on the comments, the author makes revisions and provides a newer version of code C_2 . We call the activities up to now as a review round. Note that the review process is not finished yet. The reviewers can further give suggestions on the revisions. And the author may also revise the code again. After a few review rounds, a pull request will finally be approved (changes are merged into the main branch) or discarded.

There are usually multiple commits in a pull request, and each commit may be related to multiple code changes across different files. Thus, to model the whole review process is difficult and challenging [19]. As an early step, we focus on automating the review process of a single commit. In the review round, two distinct roles are involved – contributor P_C who commits a code change ($D : C_0 \rightarrow C_1$), and reviewer P_R who gives advice and comments (R_{nl}). The goal of code review automation is to help reduce their workload. To that end, we focus on three specific code review automation tasks in both scenarios of different roles. An overview of all three tasks is shown in figure 2.

¹<https://github.com/microsoft/CodeBERT/tree/master/CodeReviewer>

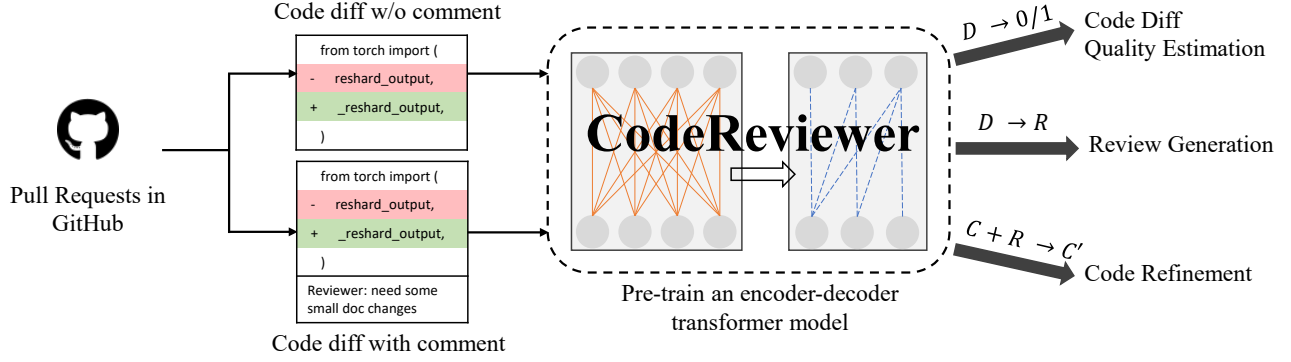


Figure 1: Overview of the workflow of CodeReviewer.

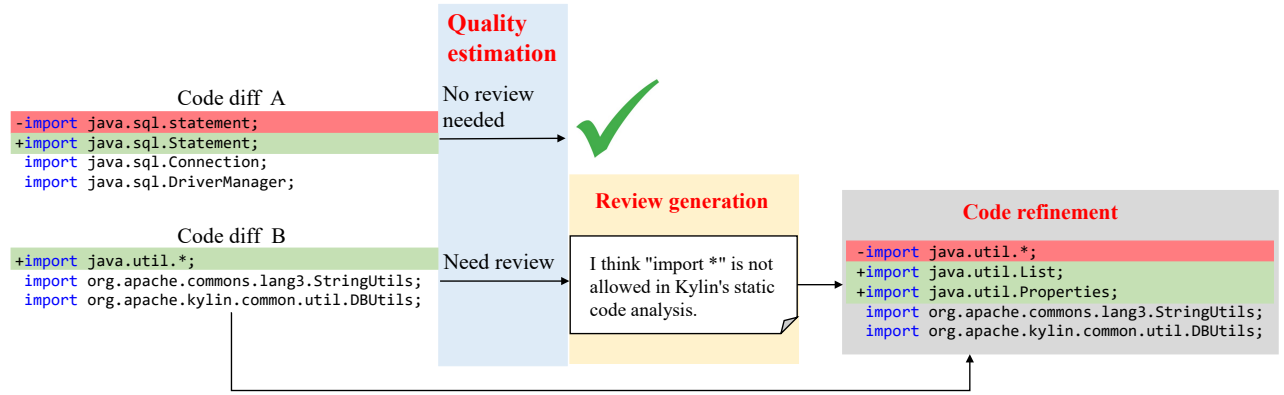


Figure 2: Overview of code review automation tasks. Three tasks are done separately.

Table 1: Summary of notations and symbols in this paper.

Notation	Definition
P_C, P_R	Author and reviewer of the code change
C_0, C_1, C_2	Different versions of source code
D	Code change between C_0 and C_1
R_{nl}	Review comment written in natural language
$\mathcal{X}, \mathcal{Y}$	Model input and output space
$c_1, c_2, \dots, c_n$	Source code tokens
$w_1, w_2, \dots, w_m$	Review comment tokens
$\mathcal{L}$	Training loss function

2.2 Code Change Quality Estimation

Code change quality estimation task is to predict whether a code change is high-quality and ready to be accepted in the review process. This is a binary classification task, i.e., $\mathcal{Y} = \{0, 1\}$. The input is a code change, i.e., $\mathcal{X} = \{D(C_0, C_1)\}$. As stated before, there are usually lots of changes across different source code files in a pull request. It takes reviewers a large amount of time to review all the changes. But it always turns out that most of the changes are minor and don't need a comment or suggestion. To improve the efficiency of code review, we define and advance to automating the code

change quality estimation task. Given the estimates, reviewers can give questionable code changes a higher priority to review and pay more attention to them, saving time and effort. Contributors can also leverage the estimates to improve low-quality code changes before submitting them to reviewers.

2.3 Code Review Generation

Code review generation is a sequence generation task. The output is a predicted review comment, i.e., $\mathcal{Y} = \{w_1, \dots, w_n\}$ where w_i is a natural language word and $n \in \mathcal{N}$ is the length of review comment. The input is still a code change, i.e., $\mathcal{X} = \{D(C_0, C_1)\}$, with its context. In some previous works [18, 40, 41], researchers use the changed code as input but not the code diff, without taking into account that review comments have to focus on the changed part. It's not recommended for reviewers to give suggestions to the code context which has not been revised. Considering the naturalness of software[21], language models can capture general statistical properties of programs due to programs tend to be repetitive and predictable. In the code review process, there are also common issues in some code changes. For example, reviewers often write "This method should be private" to suggest the contributor add a private decorator to a method. This gives us a chance to learn the general code review patterns and generate comments automatically to lighten the burden of reviewers. In the real-world scenario, the

model generates few review comment candidates and the reviewers may choose an ideal one from them, free from writing comments manually.

2.4 Code Refinement

In the code refinement task, the model takes as input both the original code C_1 written by the contributor and the review comment R_{nl} from the reviewer, and targets to generate the revised version code C_2 implementing the requirements mentioned in R_{nl} . Different from the code refinement task in other software development scenarios where only source code is taken as input [39], we design specifically for the code review process to utilize the review comment as guidance for refining. Code refinement is a task designed for assisting contributors. When they open pull requests and get review comments as feedback, the model helps to revise the submitted code automatically based on the comments.

2.5 Data Format

All three tasks are related to source code or code changes. In the code review process, there are usually multiple related files and functions [19]. A source code file or a function/method is too large to process, because there may be multiple review comments and code revisions spreading across a file or function/method. Instead, we define the inputs of the three tasks at diff hunk level.

In pull requests, the original version and revised version of code are shown in a special format: diff (Figure 2). The diff file is generated by comparing two files before and after the change. The diff tool finds sequences of lines common to both files, interspersed with groups of differing lines called hunks. Specifically, a diff hunk is a sequence of source code lines surrounded by a few unchanged lines. Diff hunk includes the lines deleted from the original file and added to the new file. The diff format is frequently used by programmers to show the changes in the code clearly. Moreover, the diff format is an efficient representation for code changes because the unchanged lines occur only once, and the changes are aligned with “-” and “+” tags at the beginning of each line. In the first two tasks, we formulate the input as a diff hunk representing the code change. In the code refinement task, we extract the input lines (C_1) and output lines (C_2) from the revision diff hunk.

3 CODE REVIEW DATASET

Nowadays, many developers use collaborative software development platforms like GitHub² and Gerrit³ not only to share their code but also to perform code review activities. GitHub platform allows contributors to make all these review details publicly available, including the exact content of code changes, review comments and their authors, and happening time in each pull request, which makes it possible to collect and analyze code review data from open-source projects. Hence, we build our CodeReview dataset based on pull requests data collected from open-source projects, covering nine programming languages, in GitHub.

²<https://github.com/>

³<https://www.gerritcodereview.com/>

3.1 Project Selection

To ensure the quality of our dataset, we collect pull request data from publicly available high-quality open-source repositories. We first sort the projects by “popularity” as indicated by the number of stars. To improve the generalizability of the models trained on our dataset, we collect projects in nine of the most popular programming languages in GitHub, including C, C++, C#, Go, Java, JavaScript, PHP, Python, and Ruby. Then, we keep the top 10,000 projects for each of the nine programming languages and remove those projects that do not explicitly permit the re-distribution of their data. To be specific, all repositories with popular licenses such as Apache-2.0 license and MIT license⁴ are kept. If the contributors write in the license file that they allow for re-distribution of their data, their projects are also collected. In order to ensure the quality of the project and acquire code review data as much as possible, we further sort the projects by the number of pull requests and filter out projects with less than 1,500 pull requests. This process keeps only active projects with many contributors and removes repositories that are forked from other projects as the pull request number is not inherited. Then we start crawling pull request information of the projects.

3.2 Review Data Collection

We use GitHub REST API to collect pull requests data from the projects. GitHub API offers an HTTP request-based API for accessing repository information. By sending HTTP requests to GitHub, we can access the branches, commits, pull requests, code diff, review comments, etc in Json format conveniently. Many researchers have used GitHub API to collect and analyze pull request data in their work [20, 35]. To advance the research of code review, Heumüller et al. [20] develop the ETCR infrastructure for mining code review datasets from any GitHub project which practices pull-request-based development. ETCR tool can be set up requiring only a GitHub API key and repository name. The ETCR uses a Kotlin-based web crawler and works in four stages to collect data of pull requests, commits, comments and source files via the GitHub API. All the data is stored in a relational database and thus can be accessed conveniently. We first use the ETCR tool [20] to collect meta-information of pull requests and review comments, including the git commit hash and changed file name related to the comments. Using the meta-information, we can further query GitHub API to get code changes (including the original file, new file, and the code diff) corresponding to the review comments. These code changes and comments make up our CodeReview data. By now, we have collected all required data for building datasets for the three downstream tasks.

3.3 Dataset Construction

The pre-training dataset is a set of code changes with or without review comments. However, it requires further data processing to build datasets for the three downstream tasks: ❶ Code change quality estimation: All commented code changes are regarded as suspicious code that introduces bugs or conflicts with code specifications. Other code changes without comments are labeled as correct. As the number of code changes without comments is about

⁴Apache-2.0, GPL-3.0, MIT, BSD-2.0, BSD-3.0, BSL-1.0, GPL-2.0 license, etc.

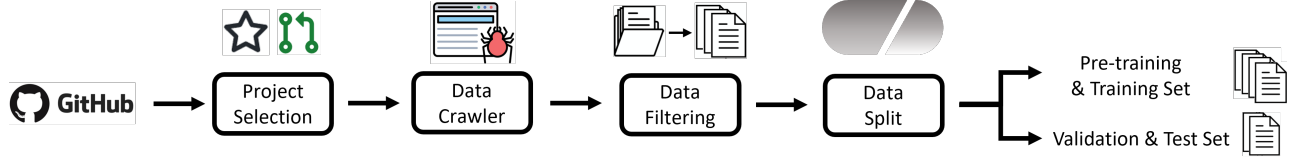


Figure 3: The process of building our dataset.

2-3 times the number of commented code changes, we perform random down-sampling on the changes without review comments to build a balanced dataset. ② Review comment generation: Code changes with review comments will be used for this task. We filter out those comments written by the code changes author. When there is more than 1 comment related to a diff hunk, we only keep the earliest one. ③ Code refinement: We traverse each pull request in the projects. For each commented code change, we check all the commits in this pull request to find whether there is a later commit that updates this part of code again. To be specific, when the reviewer writes a comment on the code diff $D : C_0 \rightarrow C_1$ and the revised source code lines in C_1 are further modified to a newer version C_2 in a later commit, we assume that the comments on the early change helped the contributor to make the later updates. Then the triplets (C_1, R_m, C_2) are collected to build the code refinement dataset. In some cases, a comment is related to multiple future code revisions or multiple comments contribute to a revision together. All these samples are removed from the dataset because modeling the interactions between multiple comments and multiple changes is beyond the discussion of this paper. Even though we collect data from projects with high star numbers and PR numbers, the dataset quality still varies, especially when it comes to review comments. To filter out low-quality comments, we clean the pre-training and downstream datasets carefully following the steps described in Appendix (attached as supplementary material).

3.4 Data Split

We use the CodeReview data to build the pre-training dataset and the benchmark dataset for the three downstream tasks. To prevent the dataset from information leakage, we split the data in project level. The code repositories with more than 2,500 pull requests are used to build the pre-training dataset and training set of the benchmark dataset. Other code repositories with [1, 500, 2, 500) pull requests are used to build the validation and test dataset.

4 CODEREVIEWER

In this section, we describe our CodeReviewer model in detail, including the model architecture, the input-output representations of the model, and the pre-training tasks designed for code review.

We develop CodeReviewer based on Transformer [42] and design four pre-training tasks related to the code review process to improve the model’s capacity for automating code review activities.

4.1 Model Architecture

The CodeReviewer is an encoder-decoder model based on Transformer [42]. We adopt the same architecture as Text-To-Text-Transfer Transformer (T5) model [31]. The CodeReviewer model consists of

12 Transformer encoder layers and 12 decoder layers. There are 12 attention heads in each layer and the hidden size is 768. The total parameter size of the model is 223M.

We initialize CodeReviewer with the parameters of CodeT5 [43]. We further pre-train the model with our four pre-training tasks. Once pre-trained, CodeReviewer is finetuned and evaluated on the downstream tasks respectively.

4.2 Input-Output Representations

CodeReviewer takes different inputs and outputs for different tasks. For code understanding tasks or comment generation tasks, the model takes code diff as input. For the code refinement task, the model takes as input both the original source code and the review comment.

Following the standard way of input processing in Transformer, the input is treated as a token sequence. We use the same RoBERTa [26] tokenizer as CodeT5 to split the source code and review comment to the tokens. A special token $[CLS]$ is prepended to the sequence, forming the input as $\{[CLS], c_1, c_2, \dots, c_n\}$ where c_i is source code token and n is the sequence length. To help our model understand the diff format better, the special line tags “-” and “+” in diff file indicating line deletion and line insertion are replaced with special tokens $[DEL]$ and $[ADD]$. We also insert a $[KEEP]$ before each unchanged line. When there are both source code tokens and review comment tokens in the input, a $[MSG]$ token is inserted to separate the two sequences. Thus the input is $\{[CLS], c_1, c_2, \dots, c_n, [MSG], w_1, w_2, \dots, w_m\}$.

The model outputs consist of two components: token representations from the encoder and generated token sequence from the decoder. For classification tasks, the representation of $[CLS]$ token is used to generate the predictions. For sequence generation tasks, the decoder outputs the predicted token sequence.

4.3 Pre-training Tasks

The key challenge of automating code review activities is to understand code changes and capture the relationship between code changes and corresponding review comments. Therefore, we design four pre-training tasks to improve the abilities of CodeReviewer.

4.3.1 Diff Tag Prediction. As mentioned in Section 2.5, diff is a special format containing rich information in code changes. Compared with encoding both the original and new version code, the diff format prevents duplicates of unchanged lines. Thus, learning code diff as a special format of source code is critical for the model in the code review scenario.

We use Diff Tag Prediction (DTP) task to equip the model with the ability to understand the special line tags in code diff. Note that we replace “+” and “-” tags at the beginning of diff lines by special

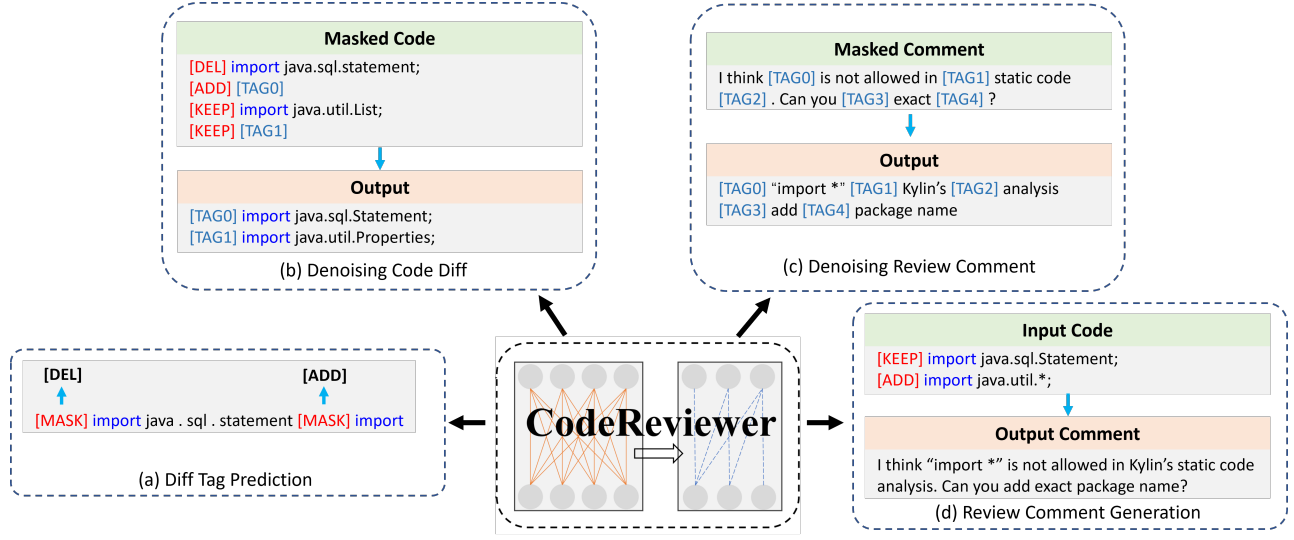


Figure 4: Pre-training tasks of CodeReviewer.

tags `[ADD]`, `[DEL]` and involve `[KEEP]` in the model input. In DTP training, the three kinds of special tags are replaced by `[MASK]` tokens in the model input. We train our model to predict special tokens `[ADD]`, `[DEL]` or `[KEEP]` at corresponding positions. By training on DTP, the model learns to distinguish whether a line is unchanged or updated. This helps CodeReviewer to understand the code diff format. Concretely, model predicts a probability distribution $\mathbf{p}^{(i)} = (p_0^{(i)}, p_1^{(i)}, p_2^{(i)})$ for the i -th masked tag position and a standard cross-entropy loss is used to train our model:

$$\mathcal{L}_{DTP} = - \sum_i \left(y_0^{(i)} \log p_0^{(i)} + y_1^{(i)} \log p_1^{(i)} + y_2^{(i)} \log p_2^{(i)} \right) \quad (1)$$

4.3.2 Denoising Objective. Denoising objective is an unsupervised pre-training objective introduced by Lewis et al. [24] to acquire a general understanding of a corpus. The original denoising objective randomly masks spans in the input sentence with a corrupted rate of 15% and predicts these spans. By learning to predict masked spans, the model gains general knowledge about the corpus distribution.

To better understand code diff and review comments, we design two denoising objectives for CodeReviewer: denoising code diff (DCD) and denoising review comment (DRC). In the DCD task, we randomly select 15% code lines and mask them. We corrupt inputs on line-level but not span-level in order to keep the integrity format of code diff. It's worth noting that to reduce the difficulty of this task, the line tags (`[ADD]`, `[DEL]` and `[KEEP]`) are preserved to inform the model whether a line is updated or removed. The DCD task aims to help the model learn the distribution of code changes, augmenting both the encoder and decoder. Encoder producing a better representation of code changes benefits both understanding and generation tasks. Pre-training the decoder on DCD helps the model generate better source code in the code refinement task. In DRC task, the model is given a corrupted review comment as input and is required to recover the masked spans. Specifically, we randomly mask spans with 20% corrupted rate and train the

decoder to generate them. Training with the DRC task is expected to benefit comment-related tasks. We describe the loss of DCD and DRC tasks as:

$$\mathcal{L}_{DCD} = \sum_{t=1}^k -\log P_{\theta}(c_t | \mathbf{c}^{\text{mask}}, \mathbf{c}_{<t}) \quad (2)$$

$$\mathcal{L}_{DRC} = \sum_{t=1}^k -\log P_{\theta}(w_t | \mathbf{w}^{\text{mask}}, \mathbf{w}_{<t}) \quad (3)$$

where $\mathbf{c}^{\text{mask}}$, $\mathbf{w}^{\text{mask}}$ are masked code diff and masked review comment, and $\mathbf{c}_{<t}$, $\mathbf{w}_{<t}$ are the span sequence of code diff and review comment generated so far.

4.3.3 Review Comment Generation. In each pre-training task mentioned above, only one modal is involved, which means that the model only learns to understand source code or review comments in one task. However, the most challenging part of code review is to capture the relationship between code changes and review comments. To equip our model with this capability, we utilize bi-modal data (code changes in programming languages and related review comments in natural language) in the pre-training task. For generalizability, we use a simple conditional generation task: review comment generation (RCG). In RCG, the model is given a code change as input and asked to generate the review comment written by the human reviewer. We use the negative log-likelihood loss:

$$\mathcal{L}_{RCG} = \sum_{t=1}^k -\log P(w_t | \mathbf{c}, \mathbf{w}_{<t}) \quad (4)$$

where $\mathbf{c}$ is code change and $\mathbf{w}_{<t}$ is the comment generated so far.

4.4 Fine-Tuning

We group all downstream tasks into classification tasks and generation tasks. For classification tasks such as code change quality estimation, we only use the pre-trained encoder. The representation in the last layer of the special token `[CLS]` at the beginning h_0 is

fed into a linear classifier to produce the prediction. For generation tasks such as code refinement, the entire pre-trained encoder-decoder model is used. A standard token negative log-likelihood loss is leveraged to optimize the probability of the target sequence.

5 STUDY DESIGN

To investigate the performance of our CodeReviewer model on the code review tasks, we perform a large-scale study to answer the following research questions (RQ):

RQ1: How does CodeReviewer perform on the code change quality estimation task? We provide as input to CodeReviewer a code change $D_1 : C_0 \rightarrow C_1$, and ask the model to give a binary prediction whether the code change needs a review.

RQ3: How does CodeReviewer perform on the review generation task? In this RQ, CodeReviewer is also provided a code change $D_1 : C_0 \rightarrow C_1$, but we assess the ability of CodeReviewer to generate a natural language comment R_{nl} as the reviewers would do.

RQ3: How does CodeReviewer perform on the code refinement task? Given a few lines of source code submitted for code review and the feedback from reviewers written in natural language, the model is required to refine the submitted code to implement the requirements of the review comments.

In RQ1-RQ3, we evaluate CodeReviewer on different code review tasks. To assess the contribution of each pre-training task and our multilingual dataset, we further investigate RQ4 and RQ5.

RQ4: What role does each pre-training task play in CodeReviewer? In RQ1-RQ3, the evaluated CodeReviewer model is pre-trained on all four pre-training tasks before being applied to the downstream tasks. We remove the pre-training tasks one by one and evaluate the resulting models again to expose the influence of each pre-training task on different downstream tasks.

RQ5: Can multilingual dataset benefit model performance on understanding single programming language? Existing research show that multilingual training dataset can benefit for model performance compared with monolingual dataset on neural machine translation and code translation [8, 48], especially for low-resource languages. We evaluate the performance of CodeReview pre-trained and fine-tuned on monolingual dataset and compare it with the model trained on the full dataset to show the influence of multilingual dataset.

Dataset We build our pre-training dataset and 3 downstream task datasets as described in Section 3.3. Table 2 summarizes the statistics of the pre-training dataset. For benchmark datasets, details are shown in Table 3.

5.1 Baseline Models

To demonstrate the superiority of our multilingual code review related pre-training dataset and carefully designed pre-training tasks, we compare our CodeReviewer model with three baselines, including a state-of-the-art (SOTA) model architecture Transformer [42] trained from scratch and two pre-trained models: T5 for code review [40] and CodeT5 [43].

Table 2: Statistics of pre-training dataset.

Language	Meta Info			Data #	
	Project	PRs [†]	Data Size	w/o comment	w/ comment
Python	195	1,451k	72.8G	887k	518k
Java	175	1,073k	54.8G	876k	467k
Go	146	951k	40.4G	728k	410k
C++	133	999k	82.1G	474k	202k
JavaScript	194	1,354k	30.6G	425k	293k
C	77	441k	135.4G	292k	110k
C#	77	463k	28.2G	324k	199k
Php	92	574k	16.0G	215k	157k
Ruby	72	626k	3.8G	90k	126k
Total	1,161	7,933k	463.2G	4,311k	2,481k

[†] Pull request numbers of the projects in total.

Table 3: Statistics of benchmark datasets.

Dataset	Train #	Valid #	Test #	LOC
Quality Estimation	~266k	~31k	~31k	~11M
Review Comment Generation	~118k	~10k	~10k	~1.8M
Code Refinement	~150k	~13k	~13k	~1.3M

Transformer. Transformer [42] is a SOTA model architecture for many classification and generation tasks. The key component of Transformer is the multi-head attention module and the parameterized linear transformation layers. We use the same model setting as CodeT5-base, with a 12-layer encoder and a 12-layer decoder.

T5 for code review. Tufano et al. [40] attempt to use pre-trained model for code review automation. They pre-train a small version of T5 model with the denoising objective proposed by Raffel et al. [31] on their own dataset including Stack Overflow dumps and CodeSearchNet [22]. The model consists of 61M parameters, with 6 layers for both encoder and decoder. We refer to this model with T5 for convenience later.

CodeT5-base. CodeT5 is a SOTA unified pre-trained encoder-decoder model for code understanding and generation tasks proposed by Wang et al. [43]. They propose to pre-train CodeT5 with identifier-aware denoising tasks and bimodal dual generation objective, which makes CodeT5 the SOTA model for multiple code-related downstream tasks including code summarization, code generation, code translation and code refinement. The base version of CodeT5 consists of 12 encoder layers and decoder layers with the parameter size 220M. We use CodeT5 to refer to this model.

5.2 Evaluation Metrics

We provide a brief description of the evaluation metrics used for the three downstream tasks in this section.

Code change quality estimation. It is a binary classification task. We use accuracy, precision, recall, and F1 to evaluate the model predictions. Note that when computing the later 3 metrics, the code changes with issues (requires for comments and updates) are treated as the positive class.

Review comment generation. We compute the BLEU (Bilingual Evaluation Understudy) [30] score of the predictions. BLEU is

widely used to assess the quality of automatically generated text in generation tasks such as machine translation and code generation. We use the BLEU-4 variant, which computes the overlap of n -grams between the generated text and the reference text with n from 1 up to 4. As review comments are diverse and non-unique, we further apply human evaluation on generated comments from two perspectives: information and relevance. ❶ Information: in this metric, we evaluate how informative the comment is for the contributor to revise the source code. Comments like “declare this variable as private” are more informative than those like “why do we need this?”. ❷ Relevance: in this metric, we evaluate to what extent the review comment is related to the corresponding code change. The comments point out the issues in the code changes will get high scores, while those not related or disobey the logic in the code changes should get low scores. The comments are labeled with a 1-5 score for each metric. We describe in detail of the two metrics in the Appendix.

Code refinement. For this task, we compute the BLEU score between generated code and target code and the exact match (EM) rate. BLEU only evaluates the similarity between the prediction and the target, while a small change in source code can result in compile error and execution error. So exact match is a more important metric in this task. Only if a prediction is exactly the same as the target, will it be considered as correct.

5.3 Implementation Details

We implement our model with the popular deep learning development framework PyTorch⁵ and the python package transformers developed by HuggingFace⁶. We pre-train our CodeReviewer model with 2 DGX-2 servers with 16 NVIDIA V100-32G GPUs on each server. The learning rate and batch size in the pre-training stage are set to 0.0002 and 768. We use AdamW optimizer with linear warmup to optimize the model for 250k steps. The warmup step number is set to 2500. When fine-tuning our CodeReviewer and baseline models on the three downstream tasks, we use batch size 72 and learning rate 0.0003. Each experiment on downstream tasks is performed on a server with 4 V100 GPUs. When evaluating the review comment generation task and the code refinement task, we use beam search with size 10.

6 RESULTS ANALYSIS

In this section, we answer each research question based on our experiment results. We first focus on three research questions concerning our model’s performance in each task and how it compares to other state-of-the-art baseline models. We further demonstrate the effects of our pre-training tasks and multilingual dataset.

6.1 RQ1: Performance on Code Change Quality Estimation

Table 4 shows the results on the code change quality estimation task. From the table, we can see that whether the baselines are trained from scratch or pre-trained models, CodeReviewer outperforms

Table 4: Results on code change quality estimation.

Model (Layers #)	Precision	Recall	F1	Accuracy
Transformer (12)	74.50	46.07	56.93	65.16
T5 (6)	70.82	57.20	63.29	66.82
CodeT5 (12)	70.36	58.96	64.16	67.07
CodeReviewer (12)	78.60	65.63	71.53	73.89

them significantly on all four metrics. Specifically, our CodeReviewer model improves F1 and accuracy by 8.24% and 7.07% compared with T5. The improvement over CodeT5 is also over/about 7%, which demonstrates that our pre-training tasks help CodeReviewer understand code changes better. Besides, the performance of Transformer trained from scratch is inferior to the other three models, indicating the importance of pre-training.

6.2 RQ2: Performance on Review Generation

Table 5 shows the results on the review generation task. CodeReviewer produces a higher BLEU score than the baseline models. However, the BLEU score of our model is still lower than 10, indicating it is a hard task. As mentioned before, review comments are diverse and non-unique. Referring to the example illustrated in Figure 5, our model prediction conveys similar intent as the ground truth. But their words differ greatly so that the BLEU score is lower than Codex. To investigate the models’ performance better, we perform a human evaluation on their predictions.

We firstly choose 300 random samples in Java and Python from the test set and manually select 100 of them with high-quality code changes and review comments. For the selected samples, we invite 6 professional sophisticated programmers to score the reference comment and generated comments of each model as described in Section 5.2. Results are listed in Table 5. Our model improves the information score and relevance score of the generated comments by about 20% relatively compared with the baseline models, which means comments generated by CodeReviewer are more useful and adequate. The performance of T5 is inferior to the other models, including the Transformer baseline. Because the parameter size of T5 (61M) is about 1/4 of the other models (223M). We conjecture that it is too challenging for a small model to generate the review comments.

In this task, we also conduct a qualitative analysis of the world’s largest language model on source code, Codex [7], which has been proven to be the most powerful code generation model and capable of zero-shot task transfer. Since we don’t get access to the model and can’t fine-tune it, we alternately use GitHub Copilot [15] which is powered by Codex. By providing several code diff and review comment pairs as prompt, we find that Codex can’t generate any meaningful reviews but copying comments in the given examples. Figure 5 shows two examples of outputs generated by different models including Codex.

⁵<https://pytorch.org/>

⁶<https://huggingface.co/>

<pre> public class DockerOptions { for (int i = 0; i < maxContainerCount; i++) { node.add(caps, new DockerSessionFactory(clientFactory, docker, image, caps)); } } - LOG.info(String.format(+ LOG.finest(String.format("Mapping %s to docker image %s %d times", caps, name, </pre>	
Ground truth: This change prevents a user understanding how their server is configured. Best to leave at `info` level.	
T5: I don't think this change is needed.	BLEU:4.93
CodeT5: Why do we need this change?	BLEU:2.97
Codex: I think this is a good example of how to use the docker	BLEU:8.83
CodeReviewer: This is an important debugging help and shouldn't be lower than the default visible INFO.	BLEU:7.97
<pre> public class MockExecutorLoader implements ExecutorLoader { public void unassignExecutor(int executionId) throws ExecutorManagerException { executionExecutorMapping.remove(executionId); } } + @Override + public List<ExecutableFlow> fetchRecentlyFinishedFlows(long lifeTimeMs) + throws ExecutorManagerException { + return null; + } </pre>	
Ground truth: I think we should return an empty list instead of null here.	
T5: I don't think we need this.	
CodeT5: I don't think this class belongs here.	
Codex: I think this is a good idea.	
CCoderReviewer: return empty list instead?	

Figure 5: Two examples of the review generation task. The Codex output is obtained by Copilot.

Review comment: flip the conditions, null check should be done before	
<pre> T5: try { wallpaperManager.setBitmap(bitmap); ViewUtil.showLongToast(context, context.getString(R.string.wallpaper_set_successfully)); - if (progressDialog.isShowing() && progressDialog != null) { + if (progressDialog.isShowing()) { progressDialog.dismiss(); } } catch (IOException e) { ... } </pre>	
<pre> CodeT5: try { wallpaperManager.setBitmap(bitmap); ViewUtil.showLongToast(context, context.getString(R.string.wallpaper_set_successfully)); - if (progressDialog.isShowing() && progressDialog != null) { + if (progressDialog.isShowing() progressDialog != null) { progressDialog.dismiss(); } } catch (IOException e) { ... } </pre>	
<pre> CodeReviewer: try { wallpaperManager.setBitmap(bitmap); ViewUtil.showLongToast(context, context.getString(R.string.wallpaper_set_successfully)); - if (progressDialog.isShowing() && progressDialog != null) { + if (progressDialog != null && progressDialog.isShowing()) { progressDialog.dismiss(); } } catch (IOException e) { ... } </pre>	

Figure 6: An example of the code refinement task. To make clear, we show the code diff between the inputs and model outputs instead of their original form.

6.3 RQ3: Performance on Code Refinement

Table 6 shows the results on the code refinement task. The Naïve-Copy method directly copies the input code as the refinement result. It produces a decent BLEU score but 0.00 Exact Match (EM).

CodeReviewer successfully generates the repaired code exactly the same as ground truth for more than 30% cases, which is two times as the result of T5 and 25% more than CodeT5 relatively, demonstrating the superior ability of CodeReviewer to understand the

Table 5: Results on review comment generation. The perfect scores of two metrics in human evaluation set are both 5.

Model (Layers #)	Test set	Human evaluation set	
	BLEU	Information	Relevance
Transformer (12)	4.76	3.08	2.50
T5 (6)	4.39	2.54	1.62
CodeT5 (12)	4.83	3.03	2.40
CodeReviewer (12)	5.32	3.60	3.20

Table 6: Results on code refinement.

Model (Layers #)	BLEU	EM
NaïveCopy	58.75	0.00
T5 (6)	77.03	15.08
CodeT5 (12)	80.82	24.41
CodeReviewer (12)	82.61	30.32

Table 7: Ablation study on code change quality estimation.

Model	Precision	Recall	F1	Accuracy
CodeReviewer	78.60	65.63	71.53	73.89
w/o CMT	77.14	66.35	71.34	73.35
w/o DTP	79.59	62.64	70.10	73.29
w/o DCD	79.87	60.94	69.13	72.80

review comments and refine code based on them. The BLEU score of CodeReviewer is also higher than T5 and CodeT5. Figure 6 shows an example of a perfect prediction generated by our model. Note that the Transformer model failed to converge in 10 epochs, producing about 0 BLEU score, so the result is not listed in the table.

6.4 RQ4: Influence of Pre-training Tasks

We demonstrate the superior performance of CodeReviewer on different downstream tasks in RQ1-RQ3, proving that the well-designed pre-training tasks benefit for automating code review activities. In this RQ, we dive deeper and expose the contribution of each pre-training task. We further pre-train three models, each of which is trained with some pre-training tasks removed, and evaluate their performance on the code change quality estimation task. The models CodeReviewer w/o DTP, CodeReviewer w/o DCD and CodeReviewer w/o CMT represent CodeReviewer trained without the Diff Tag Prediction task, the Denoising Code Diff task, and the other two comment related tasks (Denoising Review Comment and Review Comment Generation) respectively.

Table 7 shows the results. The drop of the model performance demonstrates the importance of the pre-training tasks. For the code change quality estimation task, all the pre-training tasks help our model understand the code diff better. But the diff tag prediction task and the denoising code diff task are more important. The results are consistent with our motivation in Section 4.3.

Table 8: Ablation study of multi-lingual dataset on the code change quality estimation task.

Metric	Java		C#		Ruby	
	Multi	Single	Multi	Single	Multi	Single
Accuracy	74.04	72.45	74.80	72.21	82.70	79.92
F1	70.53	69.34	76.52	75.53	89.23	88.10

6.5 RQ5: Influence of Multilingual Dataset

In the previous experiments, CodeReviewer is trained on the datasets consisting of nine programming languages. To investigate whether multilingual datasets help our model better understand a single programming language, we build monolingual datasets for Java, C# and Ruby language, respectively. Java represents popular languages and Ruby represents the low-resource languages as shown in Table 2. For each of the three languages, we pre-train and finetune the CodeReviewer on the monolingual datasets and compare the performance on the code change quality estimation task with the original CodeReviewer pre-trained and finetuned on full multilingual datasets. The results are listed in Table 8.

The multilingual CodeReviewer outperforms the three monolingual models consistently, improving the accuracy by 2.32% and the F1 score by 1.10% on average. We conclude that our multilingual dataset benefits the CodeReviewer for understanding specific languages significantly. This reveals the superiority of our multilingual dataset over the datasets for single programming language. It also proves the broad applicability of CodeReviewer in different programming languages.

7 RELATED WORKS

7.1 Pre-training for Code-related Tasks

Deep Learning techniques have been widely adopted in software engineering research [10, 44]. In recent years, motivated by the great impact of pre-training technique in natural language (NL) processing area [6, 11, 24, 31], researchers have made attempts to investigate whether pre-trained models for programming language (PL) can further facilitate software development.

Since pre-trained models can address several downstream tasks by fine-tuning, previous works target at different scopes of code-related tasks with their models. Kanade et al. [23] pre-train CuBERT in a Python code corpus mostly for classification tasks like variable-misuse classification, wrong binary operator, etc. CodeBERT [14] and GraphCodeBERT [17] are bi-directional transformer models pre-trained on NL-PL pairs in six programming languages, the latter introduces new pre-training tasks designed for source code data flow. Both models have shown effectiveness on code-text crossing tasks like NL code search and code summarization, and other code understanding tasks like code clone detection and defect detection thanks to bi-directional attention mechanism. CodeGPT [27], GPT-C [36] and Codex [7] focus on generative tasks like code completion and code generation because they are pre-trained with decoder-only transformer architecture. As for encoder-decoder models like PLBART [2] and CodeT5 [43] and unified models like UniXcoder

[16], they can be applied for both understanding and generation tasks that take source code or text as inputs.

All the above models pay no attention to tasks in the code review process. A distinct feature of code review tasks is that code changes should be considered as inputs. Recently, Tufano et al. [40] pre-trained a T5 model for automating code review activities. However, their work is different from ours on two points. First, they just use the pre-training objectives of T5 [31] and don't take into consideration how to integrate code changes into pre-training. Second, their pre-training data is not directly related to code review, but we collect data from GitHub pull requests for pre-training.

7.2 Automating Code Review Activities

As indicated by previous empirical studies, code review is an important part in the software development lifecycle and involves a significant amount of effort and time of reviewers [5, 33]. Researchers are paying more attention to automating code review activities, including reviewer recommendation [38, 47], comment location prediction [19, 34], review comment recommendation [18, 35] and code refinement [41].

Thongtanunam et al. [38] reveal that 4%-30% of reviews have code reviewer assignment problems. Thus, they propose a file location-based tool RevFinder to recommend appropriate code reviewers. To tackle the same problem, Zanjani et al. [47] design the system cHRev which utilizes code review histories to recommend reviewers for a code change. While these researchers aim at improving code review at an early stage, others are devoted to solving the more challenging tasks during the code review process. Shi et al. [34] propose the DACE framework based on CNN and LSTM to predict whether a hunk in the code change will be accepted by the reviewers. Hellendoorn et al. [19] use the Transformer architecture to solve this task. Furthermore, they also attempt to capture the relation of different hunks in a pull request by encoding each hunk and computing attention scores across diff hunks to fuse the information. Li et al. [25] formalize automatic code review as a multi-instance learning task, in which each hunk is an instance and the target is to predict whether a pull request will be accepted.

Other researchers focus on tasks related to review comments. To save the time reviewers spent on writing reviews related to common issues such as coding style and documentations, Gupta and Sundaresan [18] propose the LSTM-based model DeepMem. DeepMem learns the relations between code changes and review comments, and recommends review comments automatically based on existing code reviews and code changes. Siow et al. [35] also recommend code reviews in a retrieval-based manner. They propose an LSTM-based multi-level embedding attentional model CORE aiming at capturing the semantic information in both source code and reviews. Tufano et al. [41] use deep learning techniques to automate a different task in code review. They train a Transformer model to revise the contributors' code to implement the requirements from the review comments. In the previous work, the researchers usually train a small model for a specific task in the code review process. Differently, we propose a large model pre-trained on four general pre-training tasks for code review, producing superior performance across three different code review tasks.

7.3 Code Review Datasets

To advance the research of automating code reviews activities, researchers also make effort to collect and publish code review datasets. Tufano et al. [41] create two datasets with 17k abstracted code changes for predicting method-level code changes. The Code Review Open Platform (CROP) by Paixao et al. [29] is a dataset based on specific Gerrit projects consisting of 507k comments. The dataset from Mukadam et al. [28] has fewer comments (106k) and provides only metadata of source code. Yang et al. [46] propose the dataset with the largest number of comments. But they also provide only metadata of source code. Compared with them, we propose the largest, multilingual code review dataset with completed information collected from over 1k GitHub repositories with more than 7.9M pull requests in total.

8 THREATS TO VALIDITY

Threats to internal validity relate to the roles played by the model architecture and hyper-parameters setting. We do a small-range grid search on learning rate and batch size settings, leaving other hyper-parameters the same as those in CodeT5 [43]. It is expected that more hyper-parameter tuning would bring more improvements.

Threats to external validity are mostly related to the dataset we collect and use in this paper. Since the dataset is collected from GitHub, it is built upon only open-source projects, not industrial projects. Besides, code review is often not done by a single reviewer but multiple reviewers, and they could give different comments from different perspectives. But we only collect a single comment for a code change, leading a bias in our dataset.

Threats to construct validity include the rationality of evaluation metrics. We argue that the BLEU score which is widely used in text generation tasks is not suitable for the review generation task. So we conduct a human annotation to better evaluate the results.

9 CONCLUSION

In this paper, we focus on pre-training techniques for automating code review activities. We start by formulating three tasks in the code review process with the code change format. We collect and organize a large-scale dataset from GitHub for code review pre-training and a benchmark for evaluation on the three tasks. It is the largest dataset in code review scenario, covering nine of the most popular programming languages in GitHub. Based on this, we introduce CodeReviewer, a transformer-based encoder-decoder model pre-trained on our dataset with four designed pre-training tasks for code review. The experimental results demonstrate that our model outperforms the state-of-the-art models pre-trained with source code in all three tasks.

REFERENCES

- [1] A Frank Ackerman, Priscilla J Fowler, and Robert G Ebenau. 1984. Software inspections and the industrial production of software. In *Proc. of a symposium on Software validation: inspection-testing-verification-alternatives*. 13–40.
- [2] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified Pre-training for Program Understanding and Generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2655–2668.

- [3] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *35th International Conference on Software Engineering, ICSE '13, San Francisco, CA, USA, May 18–26, 2013*, David Notkin, Betty H. C. Cheng, and Klaus Pohl (Eds.). IEEE Computer Society, 712–721. <https://doi.org/10.1109/ICSE.2013.6606617>
- [4] Moritz Beller, Alberto Bacchelli, Andy Zaidman, and Elmar Jürgens. 2014. Modern code reviews in open-source projects: which problems do they fix?. In *11th Working Conference on Mining Software Repositories, MSR 2014, Proceedings, May 31 – June 1, 2014, Hyderabad, India*, Premkumar T. Devanbu, Sung Kim, and Martin Pinzger (Eds.). ACM, 202–211. <https://doi.org/10.1145/2597073.2597082>
- [5] Amiangshu Bosu and Jeffrey C. Carver. 2013. Impact of Peer Code Review on Peer Impression Formation: A Survey. In *ESEM. IEEE Computer Society*, 133–142.
- [6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [8] Ting-Rui Chiang, Yi-Pei Chen, Yi-Ting Yeh, and Graham Neubig. 2021. Breaking Down Multilingual Machine Translation. *CoRR* abs/2110.08130 (2021).
- [9] Moataz Chouchen, Ali Ouni, Mohamed Wiem Mkaouer, Raula Gaikovina Kula, and Katsuro Inoue. 2021. WhoReview: A multi-objective search-based approach for code reviewers recommendation in modern code review. *Appl. Soft Comput.* 100 (2021), 106908. <https://doi.org/10.1016/j.asoc.2020.106908>
- [10] Prem Devanbu, Matthew Dwyer, Sebastian Elbaum, Michael Lowry, Kevin Moran, Denys Poshyvanyk, Baishakhi Ray, Rishabh Singh, and Xiangyu Zhang. 2020. Deep learning & software engineering: State of research and future directions. *arXiv preprint arXiv:2009.08525* (2020).
- [11] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [12] Michael E. Fagan. 2002. Design and Code Inspections to Reduce Errors in Program Development (Reprint). In *Software Pioneers*, Manfred Broy and Ernst Denert (Eds.). Springer Berlin Heidelberg, 575–607. https://doi.org/10.1007/978-3-642-59412-0_35
- [13] Michael E. Fagan. 2002. A History of Software Inspections. In *Software Pioneers*. Springer Berlin Heidelberg, 562–573.
- [14] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. 1536–1547.
- [15] Github. 2021. GitHub Copilot · Your AI pair programmer. <https://copilot.github.com/>.
- [16] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. *arXiv preprint arXiv:2203.03850* (2022).
- [17] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *ICLR*.
- [18] Anshul Gupta and Neel Sundaresan. 2018. Intelligent code reviews using deep learning. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18) Deep Learning Day*.
- [19] Vincent J. Hellendoorn, Jason Tsay, Manisha Mukherjee, and Martin Hirzel. 2021. Towards Automating Code Review at Scale. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Athens, Greece) (ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 1479–1482. <https://doi.org/10.1145/3468264.3473134>
- [20] Robert Heumüller, Sebastian Nielebock, and Frank Ortmeier. 2021. Exploit Those Code Reviews! Bigger Data for Deeper Learning. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Athens, Greece) (ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 1505–1509. <https://doi.org/10.1145/3468264.3473110>
- [21] Abram Hindle, Earl T. Barr, Mark Gabel, Zhendong Su, and Premkumar Devanbu. 2016. On the Naturalness of Software. *Commun. ACM* 59, 5 (apr 2016), 122–131. <https://doi.org/10.1145/2902362>
- [22] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet Challenge: Evaluating the State of Semantic Code Search. *CoRR* abs/1909.09436 (2019).
- [23] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and evaluating contextual embedding of source code. In *International Conference on Machine Learning*. PMLR, 5110–5121.
- [24] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2019. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461* (2019).
- [25] Heng-Yi Li, Shu-Ting Shi, Ferdian Thung, Xuan Huo, Bowen Xu, Ming Li, and David Lo. 2019. Deepreview: automatic code review using deep multi-instance learning. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 318–330.
- [26] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).
- [27] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*.
- [28] Murtuza Mukadam, Christian Bird, and Peter C Rigby. 2013. Gerrit software code review data from android. In *2013 10th Working Conference on Mining Software Repositories (MSR)*. IEEE, 45–48.
- [29] Mathieu Paixao, Jens Krinke, Donggyun Han, and Mark Harman. 2018. CROP: Linking Code Reviews to Source Code Changes. In *Proceedings of the 15th International Conference on Mining Software Repositories (Gothenburg, Sweden) (MSR '18)*. Association for Computing Machinery, New York, NY, USA, 46–49. <https://doi.org/10.1145/3196398.3196466>
- [30] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.
- [31] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *J. Mach. Learn. Res.* 21, 1, Article 140 (jan 2020), 67 pages.
- [32] Peter C. Rigby and Christian Bird. 2013. Convergent contemporary software peer review practices. In *Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering, ESEC/FSE'13, Saint Petersburg, Russian Federation, August 18–26, 2013*, Bertrand Meyer, Luciano Baresi, and Mira Mezini (Eds.). ACM, 202–212. <https://doi.org/10.1145/2491411.2491444>
- [33] Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko, and Alberto Bacchelli. 2018. Modern code review: a case study at google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2018, Gothenburg, Sweden, May 27 – June 03, 2018*, Frances Paulisch and Jan Bosch (Eds.). ACM, 181–190. <https://doi.org/10.1145/3183519.3183525>
- [34] Shu-Ting Shi, Ming Li, David Lo, Ferdian Thung, and Xuan Huo. 2019. Automatic code review by learning the revision of source code. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 4910–4917.
- [35] Jing Kai Siow, Cuiyun Gao, Lingling Fan, Sen Chen, and Yang Liu. 2020. CORE: Automating Review Recommendation for Code Changes. In *SANER. IEEE*, 284–295.
- [36] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. IntelliCode Compose: Code Generation Using Transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Virtual Event, USA) (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 1433–1443. <https://doi.org/10.1145/3368089.3417058>
- [37] Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Raula Gaikovina Kula, Norihiro Yoshida, Hajimu Iida, and Ken-ichi Matsumoto. 2015. Who should review my code? A file location-based code-reviewer recommendation approach for Modern Code Review. In *22nd IEEE International Conference on Software Analysis, Evolution, and Reengineering, SANER 2015, Montreal, QC, Canada, March 2–6, 2015*, Yann-Gaël Guéhéneuc, Bram Adams, and Alexander Serebrenik (Eds.). IEEE Computer Society, 141–150. <https://doi.org/10.1109/SANER.2015.7081824>
- [38] Patanamon Thongtanunam, Chakkrit Tantithamthavorn, Raula Gaikovina Kula, Norihiro Yoshida, Hajimu Iida, and Ken-ichi Matsumoto. 2015. Who should review my code? a file location-based code-reviewer recommendation approach for modern code review. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 141–150.
- [39] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation. *ACM Trans. Softw. Eng. Methodol.* 28, 4, Article 19 (sep 2019), 29 pages. <https://doi.org/10.1145/3340544>
- [40] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using Pre-Trained Models to Boost Code Review Automation. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 2291–2302. <https://doi.org/10.1145/3510003.3510621>
- [41] Rosalia Tufano, Luca Pascarella, Michele Tufano, Denys Poshyvanyk, and Gabriele Bavota. 2021. Towards Automating Code Review Activities. In *43rd IEEE/ACM International Conference on Software Engineering, ICSE 2021, Madrid*

- Spain, 22–30 May 2021. IEEE, 163–174. <https://doi.org/10.1109/ICSE43902.2021.00027>
- [42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *NIPS*. 5998–6008.
 - [43] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. *CoRR* abs/2109.00859 (2021).
 - [44] Cody Watson, Nathan Cooper, David Nader Palacio, Kevin Moran, and Denys Poshyvanyk. 2022. A Systematic Literature Review on the Use of Deep Learning in Software Engineering Research. *ACM Trans. Softw. Eng. Methodol.* 31, 2, Article 32 (mar 2022), 58 pages. <https://doi.org/10.1145/3485275>
 - [45] Xin Yang, Raula Gaikovina Kula, Norihiro Yoshida, and Hajimu Iida. 2016. Mining the modern code review repositories: a dataset of people, process and product. In *Proceedings of the 13th International Conference on Mining Software Repositories, MSR 2016, Austin, TX, USA, May 14–22, 2016*, Miryung Kim, Romain Robbes, and Christian Bird (Eds.). ACM, 460–463. <https://doi.org/10.1145/2901739.2903504>
 - [46] Xin Yang, Raula Gaikovina Kula, Norihiro Yoshida, and Hajimu Iida. 2016. Mining the Modern Code Review Repositories: A Dataset of People, Process and Product. In *Proceedings of the 13th International Conference on Mining Software Repositories (Austin, Texas) (MSR '16)*. Association for Computing Machinery, New York, NY, USA, 460–463. <https://doi.org/10.1145/2901739.2903504>
 - [47] Motahareh Bahrani Zanjani, Huzefa Kagdi, and Christian Bird. 2015. Automatically recommending peer reviewers in modern code review. *IEEE Transactions on Software Engineering* 42, 6 (2015), 530–543.
 - [48] Ming Zhu, Karthik Suresh, and Chandan K Reddy. 2022. Multilingual Code Snippets Training for Program Translation. (2022).